

Neste caso o módulo *Control*, implementado em software, é a entidade consumidora. O módulo *Port Expander Ticket Dispenser* (PETD) implementa a interface com o mecanismo de impressão de bilhetes (*Ticket Dispenser*), realizando a receção em série da informação enviada pelo módulo de controlo e entregando-a posteriormente ao mecanismo, conforme representado na Figura 1.

O módulo PETD recebe em série uma mensagem composta por dez (10) bits de informação, segundo o protocolo de comunicação ilustrado na Figura 2b. O primeiro bit de informação indica se o bilhete a imprimir é de ida ou ida/volta. Os quatro bits seguintes indicam a estação de origem, seguido da estação de destino por mais 4 bits de informação. O último bit de informação indica o início da impressão e disponibilização do bilhete. Só pode ser iniciada nova impressão quando o mecanismo de dispensa de bilhetes informar o Control que o processo terminou através do sinal Fn.

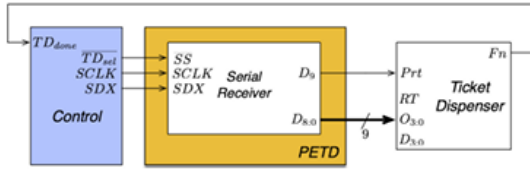
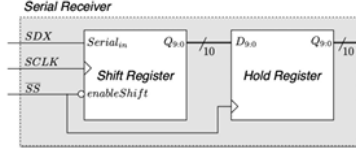


Figura 1: Diagrama de blocos do módulo *Port Expander Ticket Dispenser*

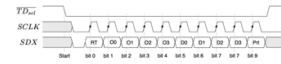
1 Serial Receiver

O módulo Serial Receiver é responsável pela receção e decodificação de dados transmitidos, convertendo a informação recebida sequencialmente num barramento paralelo de saída de dez bits. Estruturalmente, a arquitetura do componente é composta por dois sub-blocos: por um *Shift Register* que é um registo de deslocamento e um *Hold Register de 10 bits* que é um registo de contenção como mostra a Figura 2a.

O bloco *Shift Register* é responsável por receber sequencialmente os bits do sinal de entrada série SDX. O varrimento e o deslocamento interno dos dados ocorrem à cadência do SCLK sempre que o *EN shift* se encontra ativo. O bloco conta ainda com uma entrada de reset (Rst) para repor os val-



(a) Diagrama de blocos



(b) Diagrama temporal

Figura 2: Bloco *PETD*

ores do registo, disponibilizando a palavra binária resultante de dez bits na saída *Q OUT*.

Já o bloco *Hold Register10bits* implementa a lógica de retenção de dados, sendo responsável por armazenar de forma estável o vetor de dez bits proveniente do Shift Register (*D IN*). O armazenamento e a transferência da informação para o barramento de saída *D* são controlados de forma síncrona através das entradas CLK, SET e EN, sendo o fluxo global coordenado em consonância com o sinal de seleção SS. Este mecanismo garante que os dados de saída permaneçam válidos e imutáveis durante os ciclos de receção de novas tramas no barramento série.

A descrição hardware do bloco *PETD* em *VHDL* encontra-se no Anexo A.1.1.

2 Ticket Dispenser

O Ticket Dispenser recebe em 4 bits o código da estação de destino (*D3:0*), noutros 4 bits o código da estação de origem (*O3:0*) a imprimir no bilhete e ainda o bit RT que define o tipo de bilhete (ida ou ida/volta). O comando de impressão do bilhete com os códigos presentes em *D*, *O* e *RT* é realizado pela ativação do sinal de impressão (*Pprt*). Em resposta, o Ticket Dispenser ativa o sinal de término de execução (*Fn*) quando concluída a dispensa do bilhete. Este componente foi fornecido pelos docentes através da plataforma moodle.

3 Conclusões

Com a realização deste projeto, foi possível projetar e implementar com sucesso o módulo Serial Re-

ceiver em lógica programável (VHDL), cumprindo integralmente todos os requisitos especificados. A arquitetura modular implementada revelou-se uma solução robusta e adequada para receber dados por via série através do sinal SDX e disponibilizar a informação no barramento de dez bits.

A validação do circuito foi efetuada recorrendo a Testbenches e posterior teste em laboratório na placa de desenvolvimento DE10 Lite. O comportamento verificado nas simulações temporais está de acordo com o previsto, demonstrando que o varrimento e o deslocamento interno ocorrem corretamente à cadência SCLK sempre que o sinal de habilitação EN shift se encontra ativo. O mecanismo de retenção controlado pelos sinais CLK, SET e EN garantiu que os dados de saída permanecessem estáveis e válidos durante a receção de novas tramas.

A VHDL

A.1 PETD

A.1.1 PETD

```
library ieee;
use ieee.std_logic_1164.all;

entity PETD is
    port(
        SS    : in std_logic;
        SCLK  : in std_logic;
        SDX    : in std_logic;
        Rst    : in std_logic;
        RT     : out std_logic;
        Prt    : out std_logic;
        O      : out std_logic_vector(8 downto 5);
        D      : out std_logic_vector(4 downto 1)
    );
end PETD;

architecture structural of PETD is

    component SerialReceiver is
        port(
            SDX    : in std_logic;
            SCLK    : in std_logic;
            SS      : in std_logic;
            Rst     : in std_logic;
            D       : out std_logic_vector(9 downto 0)
        );
    end component;

    signal s_D_out : std_logic_vector(9 downto 0);

begin

    SerialReceiver_V: serialReceiver
        port map(
            SDX => SDX,
            SCLK => SCLK,
            SS  => SS,
            Rst => Rst,
            D   => s_D_out
        );

    Prt    <= s_D_out(0);
    D      <= s_D_out(4 downto 1);
    O      <= s_D_out(8 downto 5);
    RT     <= s_D_out(9);

end structural;
```


A.1.2 Serial Receiver

```

library ieee;
use ieee.std_logic_1164.all;

entity SerialReceiver is
    port(
        SDX      : in std_logic;
        SCLK      : in std_logic;
        SS        : in std_logic;
        Rst       : in std_logic;
        D         : out std_logic_vector(9 downto 0)
    );

end SerialReceiver;

architecture structural of SerialReceiver is

    component ShiftRegister is
        port(
            Serial_in : in std_logic;
            EN_shift   : in std_logic;
            SCLK        : in std_logic;
            Rst         : in std_logic;
            Q_OUT       : out std_logic_vector(9 downto 0)
        );
    end component;

    component HoldRegister10bits is
        port(
            CLK      : in STD_LOGIC;
            Rst      : in STD_LOGIC;
            SET      : in STD_LOGIC;
            EN       : in STD_LOGIC;
            D_IN     : in STD_LOGIC_VECTOR(9 downto 0);
            Q_OUT    : out STD_LOGIC_VECTOR(9 downto 0)
        );
    end component;

    signal s_D_in      : std_logic_vector(9 downto 0);
    signal s_not_ss    : std_logic;

begin

    ShiftRegister_V : ShiftRegister
        port map(
            Serial_in => SDX,
            EN_shift   => s_not_ss,
            SCLK        => SCLK,
            Rst         => Rst,
            Q_OUT       => s_D_in
        );

    HoldRegister_V : HoldRegister10bits
        port map(
            CLK      => SS,
            Rst      => Rst,
            SET      => '0',
            EN       => '1',
            D_IN     => s_D_in,
            Q_OUT    => D
        );

    s_not_ss <= not SS;

end structural;

```


A.1.3 Shift Register

```
library ieee;
use ieee.std_logic_1164.all;

entity ShiftRegister is
    port(
        Serial_in : in std_logic;
        EN_shift   : in std_logic;
        SCLK       : in std_logic;
        Rst        : in std_logic;
        Q_OUT      : out std_logic_vector(9 downto 0)
    );
end ShiftRegister;

architecture structural of ShiftRegister is

    component FFD is
        port(
            CLK      : in  STD_LOGIC;
            Rst      : in  STD_LOGIC;
            SET      : in  STD_LOGIC;
            D        : in  STD_LOGIC;
            EN       : in  STD_LOGIC;
            Q        : out STD_LOGIC
        );
    end component;

    signal Q_s : std_logic_vector(9 downto 0);

begin

    FFO : FFD
        PORT MAP (
            CLK  => SCLK,
            Rst  => Rst,
            SET  => '0',
            EN   => EN_shift,
            D    => Serial_in,
            Q    => Q_s(0)
        );

    FF1 : FFD
        PORT MAP (
            CLK  => SCLK,
            Rst  => Rst,
            SET  => '0',
            EN   => EN_shift,
            D    => Q_s(0),
            Q    => Q_s(1)
        );

    FF2 : FFD
        PORT MAP (
            CLK  => SCLK,
            Rst  => Rst,
            SET  => '0',
            EN   => EN_shift,
            D    => Q_s(1),
            Q    => Q_s(2)
        );

    FF3 : FFD
        PORT MAP (
            CLK  => SCLK,
            Rst  => Rst,
            SET  => '0',
            EN   => EN_shift,
            D    => Q_s(2),
            Q    => Q_s(3)
        );
```


A.1.4 Hold Register

```

LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY HoldRegister10bits IS
    PORT(
        CLK      : in  STD_LOGIC;
        Rst      : in  STD_LOGIC;
        SET      : in  STD_LOGIC;
        EN       : in  STD_LOGIC;
        D_IN     : in  STD_LOGIC_VECTOR(9 downto 0);
        Q_OUT    : out STD_LOGIC_VECTOR(9 downto 0)
    );
END HoldRegister10bits;

ARCHITECTURE structural OF HoldRegister10bits IS

    COMPONENT FFD IS
        PORT(
            CLK      : in  STD_LOGIC;
            Rst      : in  STD_LOGIC;
            SET      : in  STD_LOGIC;
            D        : in  STD_LOGIC;
            EN       : in  STD_LOGIC;
            Q        : out STD_LOGIC
        );
    END COMPONENT;

BEGIN

    FF0 : FFD
        PORT MAP (
            CLK      => CLK,
            Rst      => Rst,
            SET      => '0',
            EN       => '1',
            D        => D_IN(0),
            Q        => Q_OUT(0)
        );

    FF1 : FFD
        PORT MAP (
            CLK      => CLK,
            Rst      => Rst,
            SET      => '0',
            EN       => '1',
            D        => D_IN(1),
            Q        => Q_OUT(1)
        );

    FF2 : FFD
        PORT MAP (
            CLK      => CLK,
            Rst      => Rst,
            SET      => '0',
            EN       => '1',
            D        => D_IN(2),
            Q        => Q_OUT(2)
        );

    FF3 : FFD
        PORT MAP (
            CLK      => CLK,
            Rst      => Rst,
            SET      => '0',
            EN       => '1',
            D        => D_IN(3),
            Q        => Q_OUT(3)
        );

    FF4 : FFD
        PORT MAP (

```

A.1.5 Flip Flop

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY FFD IS
PORT(
    CLK : in std_logic;
        Rst : in STD_LOGIC;
        SET : in std_logic;
        D : IN STD_LOGIC;
        EN : IN STD_LOGIC;
        Q : out std_logic
    );
END FFD;

ARCHITECTURE logicFunction OF FFD IS
BEGIN

Q <= '0' when Rst = '1' else '1' when SET = '1' else D WHEN rising_edge(clk) and EN = '1';

END LogicFunction;
```


A.2 Ticket Dispenser (Fornecido pelos Docentes)

A.2.1 ticketDispenser

```

LIBRARY IEEE;
use IEEE.STD_LOGIC_1164.all;

-- RT bit indicates either is a one-way ticket (1) or a two-way ticket (0)
-- D - Destination Station Code
-- O - Origin Station Code
entity TICKET_DISPENSER is
    port ( RT, Prt, CollectTicket: in STD_LOGIC;
           O, D: in STD_LOGIC_VECTOR(3 downto 0);
           Fn: out STD_LOGIC;
           HEX0, HEX1, HEX2, HEX3, HEX4, HEX5: out STD_LOGIC_VECTOR(7 downto 0) );

end TICKET_DISPENSER;

architecture arq_TICKET_DISPENSER of TICKET_DISPENSER is

    component DECODERHEX

        port ( A: in STD_LOGIC_VECTOR(3 downto 0);
              ewr: in STD_LOGIC_VECTOR(7 downto 0);
              clear: in STD_LOGIC;
              HEX0: out STD_LOGIC_VECTOR(7 downto 0) );

    end component;

    signal RTtoBDC: STD_LOGIC_VECTOR(3 downto 0);

begin

    -- Activate Fn when client retrieves the printed ticket
    Fn <= CollectTicket;

    -- Convert RT bit to BDC
    RTtoBDC <= "0000" when RT = '1' else "0001";

    -- Using the six 7 segment displays available on the MAX10 Lite Board
    -- produce a valid array of numbers and letters which the client can understand
    -- Valid format example: [b0oid4]
    -- Means: "bilhete de ida e volta, da estação de origem com o código (0001) para a estação de destino com o código 0"
    -- Implementation on the six 7 displays segment, facing the board, starting from the right to the left:
    U0: DECODERHEX port map (
        A => D,
        ewr => "11111111",
        clear => not Prt,
        HEX0 => HEX4 );

    U1: DECODERHEX port map (
        -- Always d in BDC
        A => "1101",
        ewr => "11111111",
        clear => '0',
        HEX0 => HEX5 );

    U2: DECODERHEX port map (
        A => O,
        ewr => "11111111",
        clear => not Prt,
        HEX0 => HEX2 );

    U3: DECODERHEX port map (
        A => "0000",
        -- A value assignment is redundant here, since the value in ewr will overwrite it, because its different from
        -- Always o, which has no representation in BDC
        ewr => "10100011",
        clear => '0',
        HEX0 => HEX3 );

    U4: DECODERHEX port map (
        -- RT bit in BDC

```

A.2.2 int7seg

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.all;

entity INT7SEG is

    port(
        d: in STD_LOGIC_VECTOR(3 downto 0);
        ewr: in STD_LOGIC_VECTOR(7 downto 0);
        dOut: out STD_LOGIC_VECTOR(7 downto 0) );

end INT7SEG;

architecture logicFunction of INT7SEG is

begin

    -- ewr stands for enable write, which will let the user directly set values for each LED available in a 7 display segment

    dOut <=
        ewr when ewr /= "1111111" else
            "11000000" when d = "0000" else
            "11111001" when d = "0001" else
            "10100100" when d = "0010" else
            "10110000" when d = "0011" else
            "10011001" when d = "0100" else
            "10010010" when d = "0101" else
            "10000010" when d = "0110" else
            "11111000" when d = "0111" else
            "10000000" when d = "1000" else
            "10011000" when d = "1001" else
            "10001000" when d = "1010" else -- A
            "10000011" when d = "1011" else -- B
            "11000110" when d = "1100" else -- C
            "10100001" when d = "1101" else -- D
            "10000110" when d = "1110" else -- E
            "10001110"; -- F

end logicFunction;
```

A.2.3 decoderHex

```
LIBRARY IEEE;
use IEEE.std_logic_1164.all;

entity DECODERHEX is

    port ( A: in STD_LOGIC_VECTOR(3 downto 0);
           ewr: in STD_LOGIC_VECTOR(7 downto 0);
           clear: in STD_LOGIC;
           HEX0: out STD_LOGIC_VECTOR(7 downto 0) );

end DECODERHEX;

architecture logicFuntion of DECODERHEX is

    component INT7SEG

        port(
            d: in STD_LOGIC_VECTOR(3 downto 0);
            ewr: in STD_LOGIC_VECTOR(7 downto 0);
            dOut: out STD_LOGIC_VECTOR(7 downto 0) );

    end component;

    signal HEX0t: STD_LOGIC_VECTOR(7 downto 0);

begin

    U0: int7seg port map(A, ewr, HEX0t);

    HEX0 <= HEX0t when clear = '0' else "11111111";

end logicFuntion;
```

B Kotlin

B.1 Serial Emitter

Algoritmo 1: Serial Emitter

```
import isel.leic.utils.Time

object SerialEmitter {
    enum class Peripheral { LCD, TICKET }

    private const val SDX_MASK = 0x01 // 00
    private const val SCLK_MASK = 0x02 // 01
    private const val LCDSEL_MASK = 0x04 // 02
    private const val TICKETSEL_MASK = 0x08 // 03

    fun init() {
        HAL.init()
        HAL.clrBits(SDX_MASK)
        HAL.clrBits(SCLK_MASK)
        HAL.setBits(LCDSEL_MASK)
        HAL.setBits(TICKETSEL_MASK)
    }

    fun send(addr: Peripheral, data: Int) {
        when (addr) {
            Peripheral.LCD -> sendLCD(data)
            Peripheral.TICKET -> sendTicket(data)
        }
    }

    private fun sendLCD(data: Int) {
        HAL.setBits(LCDSEL_MASK)
        HAL.clrBits(SCLK_MASK)

        HAL.clrBits(LCDSEL_MASK)

        for (i in 0..9) {
            HAL.clrBits(SCLK_MASK)

            val bit = (data shr i) and 1
            if (bit == 1) HAL.setBits(SDX_MASK)
            else HAL.clrBits(SDX_MASK)

            Time.sleep(1)
            HAL.setBits(SCLK_MASK)
            Time.sleep(1)
        }

        HAL.clrBits(SCLK_MASK)
        HAL.setBits(LCDSEL_MASK)
    }

    private fun sendTicket(data: Int) {
        HAL.setBits(TICKETSEL_MASK)
        HAL.clrBits(SCLK_MASK)

        HAL.clrBits(TICKETSEL_MASK)

        for (i in 0..9) {
            HAL.clrBits(SCLK_MASK)

            val bit = (data shr i) and 1
            println("clock_${i+1}_->bit=${bit}")

            if (bit == 1) HAL.setBits(SDX_MASK)
            else HAL.clrBits(SDX_MASK)

            Time.sleep(1)
            HAL.setBits(SCLK_MASK)
            Time.sleep(1)
        }
    }
}
```

```

        HAL.clrBits(SCLK_MASK)
        HAL.setBits(TICKETSEL_MASK)

    }

    fun isBusy(): Boolean = false
}

```

B.2 Serial Receiver

Algoritmo 2: Serial Receiver

```

import isel.leic.utils.Time

const val INVALID_KEYCODE = -1

object SerialReceiver {

    private const val RXD_MASK = 0x80    // I7
    private const val RXCLK_MASK = 0x80  // O7

    private const val HALF_CLOCK_MS = 2L
    private const val SAMPLE_DELAY_MS = 2L
    private const val FRAME_END_DELAY_MS = 2L

    fun init() {
        HAL.clrBits(RXCLK_MASK)
    }

    fun rcv(): Int {
        // sem trama
        if (HAL.isBit(RXD_MASK)) return INVALID_KEYCODE

        // pequena espera para garantir start est vel
        Time.sleep(SAMPLE_DELAY_MS)

        // clock 1: deve dar 1
        pulseClock()
        Time.sleep(SAMPLE_DELAY_MS)
        if (!HAL.isBit(RXD_MASK)) return INVALID_KEYCODE

        // clocks 2..5: K0..K3
        var keyCode = 0
        for (i in 0..3) {
            pulseClock()
            Time.sleep(SAMPLE_DELAY_MS)

            if (HAL.isBit(RXD_MASK)) {
                keyCode = keyCode or (1 shl i)
            }
        }

        // clock 6: deve dar 0
        pulseClock()
        Time.sleep(SAMPLE_DELAY_MS)
        if (HAL.isBit(RXD_MASK)) return INVALID_KEYCODE

        // clock 7: deve dar 1
        pulseClock()
        Time.sleep(SAMPLE_DELAY_MS)
        if (!HAL.isBit(RXD_MASK)) return INVALID_KEYCODE

        // clock extra para fechar a trama
        pulseClock()
        Time.sleep(FRAME_END_DELAY_MS)

        // garantir que voltou a idle
        if (!HAL.isBit(RXD_MASK)) return INVALID_KEYCODE

        return keyCode
    }
}

```

```

fun waitRcv(): Int {
    while (HAL.isBit(RXD_MASK)) {
        Time.sleep(1)
    }
    return rcv()
}

private fun pulseClock() {
    HAL.setBits(RXCLK_MASK)
    Time.sleep(HALF_CLOCK_MS)
    HAL.clrBits(RXCLK_MASK)
    Time.sleep(HALF_CLOCK_MS)
}

```

B.3 TD

Algoritmo 3: Ticket Dispenser

```

object TD {

    fun init() {
        SerialEmitter.init()
    }

    fun activatePrintingTicket(roundTrip: Boolean, origin: Int, destination: Int) {
        val rt = if (roundTrip) 1 else 0
        val prt = 1

        val data =
            rt or
                (destination shl 1) or
                (origin shl 5) or
                (prt shl 9)

        SerialEmitter.send(SerialEmitter.Peripheral.TICKET, data)
    }
}

fun main() {
    TD.init()

    println("Teste1:ida,origem1,destino2")
    TD.activatePrintingTicket(false, 1, 2)
    Thread.sleep(3000)

    println("Teste2:idaevolta,origem3,destino4")
    TD.activatePrintingTicket(true, 3, 4)
    Thread.sleep(3000)

    println("Teste3:ida,origem5,destino6")
    TD.activatePrintingTicket(false, 5, 6)
    Thread.sleep(3000)

    println("Fim dos testes")
}

```